INSTITUTO POLITÉCNICO NACIONAL • UPIICSA

Ingeniería en Informática | Academia de Computación e Informática | Secuencia 7NV79

PROYECTO ROBOT CENTINELA • DOCUMENTACIÓN TÉCNICA OFICIAL

ARQUITECTURA DE BASE DE DATOS, CONTRATO JSON v1.1 Y MANUAL DE INTEGRACIÓN WEB / MÓVIL

Esquema PostgreSQL 15+ en 3FN (16 tablas), ingesta idempotente, calibración de gases MQ, lógica de sensores IR y especificación de vistas para los equipos de interfaz

JSON v1.1 • POSTGRESQL 15+ (SUPABASE) • CALIBRACIÓN MQ • 4 VISTAS SQL • RLS MULTI-HOGAR

16 TABLAS	12 CANALES	6 SENSORES IR	4 VISTAS SQL
Esquema 3FN Normalizado	Catálogo de Sensores	4 Piso + 2 Laterales	Web y Móvil Unificados

FICHA TÉCNICA Y DE CONTROL

PROYECTO
Robot Centinela — Gases y Navegación

ASIGNATURA / ÁMBITO
Internet de las Cosas (IoT) y Backend Cloud

SECUENCIA Y PERIODO
7NV79 • Octubre 2026

MOTOR DE BASE DE DATOS
Supabase (PostgreSQL 15+ en 3FN)

CHASIS Y ODOMETRÍA
Rover V7.2 (Sin baleros) • Encoders GM25-370

BATERÍA Y SENSORES IR
Li-ion 3S (11.4 V) • 4 IR Piso + 2 IR Pared

VERSIÓN DE CONTRATO
Telemetría JSON v1.1 • 4 Vistas Web/Móvil

INTEGRANTES DEL EQUIPO

1. Hernández Muñoz Ángel
Arquitectura IoT, Firmware ESP32 y Base de Datos

2. Equipo de Hardware y Control
Rover V7.2, Odometría con Encoders y Sensores IR

3. Equipo de Backend & Supabase
Esquema 3FN, RPC Idempotente y Calibración MQ

4. Equipo de Aplicación Web
Dashboard en Vivo, Gráficas y Gestión Técnica

5. Equipo de Aplicación Móvil
Monitoreo Residencial, Alertas y Estado en Tiempo Real

ÍNDICE GENERAL

Contenido del Expediente Técnico

Este documento constituye la referencia técnica oficial del Robot Centinela. Define la estructura exacta del JSON v1.1 que envía el robot, el funcionamiento interno de la base de datos en Supabase y las vistas y reglas que deben consumir los equipos de Web y Móvil.

Sección 1 Arquitectura General y Reglas de Oro del Sistema 3

1.1 Flujo Completo del Dato: Del ESP32 a las Pantallas Web y Móvil 3

1.2 Las 5 Reglas de Oro de Arquitectura (Acuerdos de Equipo) 3

Sección 2 Contrato JSON de Telemetría v1.1 y Catálogo de Sensores 5

2.1 Estructura Raíz del JSON (16 Campos) y Ejemplo Validado 5

2.2 Catálogo Maestro de los 12 Canales (channel_catalog) 8

2.3 Lógica de los 6 Sensores Infrarrojos (4 de Piso y 2 Laterales) 9

Sección 3 Ingesta Transaccional (ingest_telemetry) e Idempotencia 10

3.1 Diagrama y Secuencia Paso a Paso de la Función ingest_telemetry 10

3.2 Códigos de Respuesta del Servidor y Envío desde el Puente MQTT 11

Sección 4 Calibración de Gases MQ y Reglas Automáticas de Alerta 13

4.1 Fórmula de Conversión ADC a PPM y Parámetros por Sensor 13

4.2 Calibración de R0 en Aire Limpio y Ciclo de Alertas con Histéresis 13

Sección 5 Modelo Relacional en 3FN y Diccionario de Datos (16 Tablas) 14

5.1 Diagrama Entidad-Relación (DER) y Dominios del Esquema 14

5.2 Diccionario de Datos de las 16 Tablas de PostgreSQL 15

Sección 6 Manual Directo de Integración para los Equipos de Web y Móvil 23

6.1 Especificación de las 4 Vistas SQL Oficiales (Columnas Exactas) 23

6.2 Reglas de Interfaz (UI/UX): Cómo Mostrar Gases, Batería e IR 25

6.3 Roles RLS y Ejemplos Listos de Código (Consultas, Realtime y RPC) 25

Sección 7 Despliegue en Supabase, Pruebas Verificadas y Código SQL 27

7.1 Orden de Ejecución en Supabase y Resultados del Test (42/42 OK) 27

7.2 Scripts SQL Modulares Completos (01_schema a 05_rls) 27

SECCIÓN 1

1. Arquitectura General y Reglas de Oro del Sistema

1.1 Flujo Completo del Dato: Del ESP32 a las Pantallas Web y Móvil

El sistema Centinela opera sobre un chasis Rover V7.2 (237.5 × 157.5 × 62.5 mm, diseño con ejes impresos directos sin baleros metálicos), traccionado por dos motorreductores GM25-370 con encoders incrementales y alimentado por una batería Li-ion 3S (11.4 V nominales). El microcontrolador ESP32 recolecta las lecturas de 12 canales de sensores y publica cada 2 segundos un paquete JSON v1.1 hacia la nube.

En Supabase (PostgreSQL 15+), la función transaccional ingest_telemetry(jsonb) recibe el paquete, valida su integridad, desduplica reintentos de red, reparte los datos en las 16 tablas normalizadas en Tercera Forma Normal (3FN), interpreta las lecturas crudas de gas y evalúa automáticamente las reglas de alerta. Finalmente, las aplicaciones Web y Móvil consumen la información ya procesada a través de 4 vistas SQL.

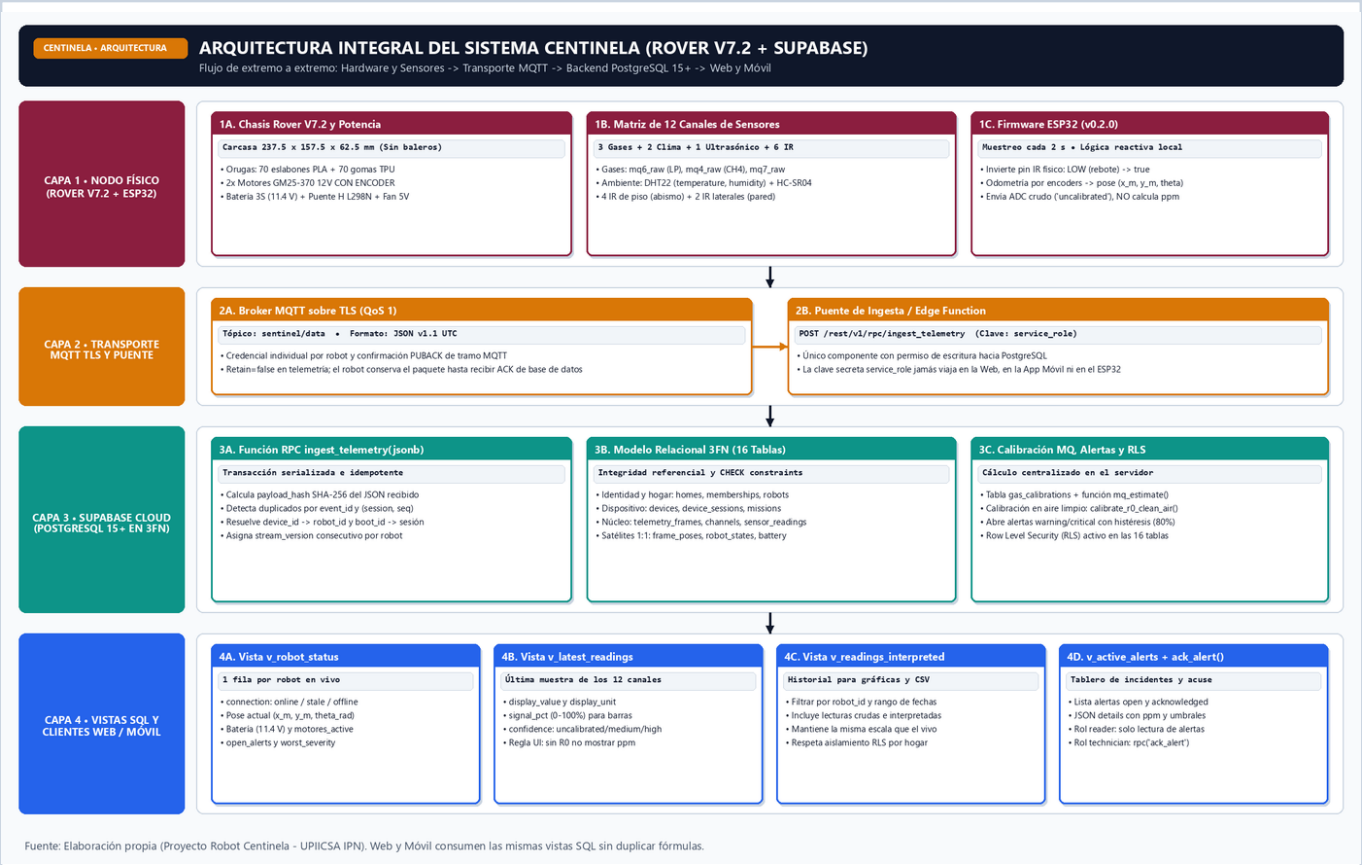


Figura 1: Arquitectura en 4 capas del Sistema Centinela: Robot ESP32, Puente MQTT, Base de Datos PostgreSQL 15+ y Clientes Web/Móvil.

1.2 Las 5 Reglas de Oro de Arquitectura (Acuerdos de Equipo)

Para evitar errores de integración entre los equipos de Firmware, Base de Datos, Web y Móvil, todo el proyecto se rige por las siguientes cinco reglas:

Regla de Arquitectura	Qué significa en la práctica	Cómo está resuelto en la Base de Datos
1. Datos Crudos en el Robot; Cálculo en la Base de Datos	El ESP32 envía lecturas crudas del ADC (0–4095) con quality = 'uncalibrated'. Nunca calcula logaritmos ni ppm en el microcontrolador.	La función SQL mq_estimate(p_raw, params) convierte el ADC a voltaje, Rs/R0, ppm y porcentaje de señal (signal_pct) dentro de PostgreSQL.
2. Cero Fórmulas en Frontend (Web y Móvil Unificados)	Los equipos de Web y Móvil NO deben programar ecuaciones de gases ni umbrales propios en JavaScript, Kotlin o Swift.	Web y Móvil consultan únicamente las 4 vistas oficiales: v_robot_status, v_latest_readings, v_readings_interpreted y v_active_alerts.
3. Lógica Positiva en los 6 IR (true = Condición Segura)	Los módulos infrarrojos entregan 0V (LOW) cuando hay rebote de luz, pero el firmware invierte la señal antes de armar el JSON.	En piso (ir_floor_*): true = Piso firme / false = Vacío (abismo). En laterales (ir_lateral_*): true = Libre / false = Pared u obstáculo.
4. Ingesta Idempotente por event_id y (session, sequence)	Si el robot o el puente MQTT reenvía el mismo paquete por reconexión WiFi, la base de datos no duplica filas ni altera las gráficas.	ingest_telemetry(p jsonb) calcula el SHA-256 del paquete y devuelve status = 'duplicate' si el event_id ya fue procesado con el mismo contenido.
5. Seguridad Multi-Hogar (RLS) con Roles reader y technician	Cada usuario autenticado solo puede leer los robots, mapas, lecturas y alertas del domicilio (home_id) al que pertenece.	Row Level Security activo en las 16 tablas. Rol 'reader' solo consulta; rol 'technician' puede reconocer alertas con ack_alert(p_alert_id).

SECCIÓN 2

2. Contrato JSON de Telemetría v1.1 y Catálogo de Sensores

2.1 Estructura Raíz del JSON (16 Campos) y Ejemplo Validado

Cada paquete enviado por el robot tiene exactamente 16 propiedades en su nivel raíz. En la siguiente tabla se especifica el tipo, obligatoriedad y destino de cada campo:

Campo en el JSON	Tipo	Carácter	Ejemplo	Función y Destino en PostgreSQL
schema_version	String	Obligatorio	"1.1"	Versión del contrato JSON. Guardado en telemetry_frames.schema_version.
event_id	UUID v4	Obligatorio (PK)	9f225860-42f2-4cd0...	ID único del paquete. Es la llave primaria telemetry_frames.id.
device_id	UUID v4	Obligatorio (FK)	bf8d0f7d-f739-4529...	ID del ESP32 (devices.id). El servidor deduce robot_id a partir de este campo.
boot_id	UUID v4	Obligatorio	b503cb0b-3866-491b...	ID generado en cada reinicio del ESP32. Crea o reutiliza la fila en device_sessions.
sequence	Bigint	Obligatorio (>=0)	1843	Contador incremental de paquetes dentro del mismo boot_id. Único por sesión.
captured_at	RFC3339 (UTC)	Obligatorio	"2026-10-11T15:24:12Z"	Hora de captura en el robot. El servidor registra además received_at = now().
monotonic_ms	Bigint	Obligatorio (>=0)	929450	Milisegundos desde el encendido del ESP32 (millis()), inmune a desajustes de hora.
clock_quality	String	Obligatorio	"synced" | "unsynced"	Indica si el ESP32 ya sincronizó hora por NTP ('synced') o no ('unsynced').
mission_id	UUID v4	Opcional (null)	0354909f-cc35-4ebc...	ID de la misión activa. Si no existe o es null, el paquete se guarda sin fallar.
map_version_id	UUID v4	Opcional (null)	ecb3df81-477f-4e72...	ID del mapa 2D sobre el que se calcula la pose. Si no existe, se guarda como null.
pose	Objeto JSON	Opcional (null)	{x_m, y_m, theta_rad...}	Coordenadas 2D calculadas con los encoders GM25-370. Alimenta la tabla frame_poses.
readings	Arreglo JSON	Obligatorio (12)	[{channel_key, ...}]	Lista de las 12 lecturas de sensores del paquete. Alimenta sensor_readings.

Campo en el JSON	Tipo	Carácter	Ejemplo	Función y Destino en PostgreSQL
battery	Objeto JSON	Obligatorio	{voltage_v: 11.4, ...}	Estado de la batería 3S. Alimenta la tabla battery_readings.
robot_state	String	Obligatorio	"moving" | "sampling"	Estado operativo reportado por el ESP32 (robot_states.reported_state).
motors_active	Boolean	Obligatorio	true | false	Indica si los motores están recibiendo PWM en ese instante.
firmware_version	String	Obligatorio	"0.2.0"	Versión del firmware del ESP32; actualiza automáticamente devices.firmware_version.

A continuación se muestra el archivo oficial JSON_estructura.json (versión 1.1) validado contra la función de ingesta:

JSON_estructura.json — Paquete Oficial de Telemetría v1.1

```
{
  "schema_version": "1.1",
  "event_id": "9f225860-42f2-4cd0-8c3a-e4e970395307",
  "device_id": "bf8d0f7d-f739-4529-9a8a-5975f5b26960",
  "boot_id": "b503cb0b-3866-491b-aebc-e164ea73d9aa",
  "sequence": 1843,
  "captured_at": "2026-10-11T15:24:12.000Z",
  "monotonic_ms": 929450,
  "clock_quality": "synced",
  "mission_id": "0354909f-cc35-4ebc-822d-bfffeaa80374",
  "map_version_id": "ecb3df81-477f-4e72-92c6-8b38f856cdb5",
  "pose": {
    "x_m": 2.15,
    "y_m": 1.40,
    "theta_rad": 0.28,
    "sigma_position_m": 0.08,
    "quality": "estimated"
  },
  "readings": [
    {
      "channel_key": "mq6_raw",
      "value": 1860,
      "quality": "uncalibrated",
      "calibration_id": null
    },
    {
      "channel_key": "mq4_raw",
      "value": 1420,
      "quality": "uncalibrated",
      "calibration_id": null
    },
    {
      "channel_key": "mq7_raw",
      "value": null,
      "quality": "not_installed",
      "calibration_id": null
    },
    {
      "channel_key": "temperature",
      "value": 24.6,
```

JSON_estructura.json — Paquete Oficial de Telemetría v1.1 (continuación)


```
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "humidity",
      "value": 48.1,
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "front_distance_m",
      "value": 0.45,
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "ir_floor_front_left",
      "boolean_value": true,
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "ir_floor_front_right",
      "boolean_value": true,
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "ir_floor_rear_left",
      "boolean_value": true,
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "ir_floor_rear_right",
      "boolean_value": true,
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "ir_lateral_left",
      "boolean_value": true,
      "quality": "valid",
      "calibration_id": null
    },
    {
      "channel_key": "ir_lateral_right",
      "boolean_value": true,
      "quality": "valid",
      "calibration_id": null
    }
  ],
  "battery": {
    "voltage_v": 11.4,
    "soc_pct": null,
    "current_a": null,
    "method_code": "voltage_only",
    "quality": "voltage_only"
  },
  "robot_state": "moving",
```

JSON_estructura.json — Paquete Oficial de Telemetría v1.1 (continuación)

```
  "motors_active": true,
  "firmware_version": "0.2.0"
}
```

2.2 Catálogo Maestro de los 12 Canales (channel_catalog)

El arreglo 'readings' del JSON transporta los 12 canales físicos del robot. Los sensores numéricos utilizan el campo 'value' (dejando 'boolean_value' en null) y los sensores infrarrojos utilizan 'boolean_value' (dejando 'value' en null):

channel_key	metric_type	unit_code	Campo Usado	Sensor Físico y Magnitud Medida	quality Inicial
mq6_raw	gas_raw	adc	value (double)	Sensor MQ-6: Gas LP (propano/butano). Lectura cruda ADC 12 bits (0–4095).	uncalibrated
mq4_raw	gas_raw	adc	value (double)	Sensor MQ-4: Gas metano / gas natural. Lectura cruda ADC 12 bits (0–4095).	uncalibrated
mq7_raw	gas_raw	adc	value = null	Sensor MQ-7: Monóxido de carbono (CO). Reservado (no instalado físicamente).	not_installed
temperature	temperature	celsius	value (double)	Sensor DHT22: Temperatura ambiental en grados Celsius (°C).	valid
humidity	humidity	percent	value (double)	Sensor DHT22: Humedad relativa ambiental en porcentaje (0–100%).	valid
front_distance_m	distance	meter	value (double)	Sensor ultrasónico HC-SR04 frontal: Distancia libre al frente en metros.	valid
ir_floor_front_left	digital	bool	boolean_value	IR de piso delantero izquierdo (true = Piso seguro, false = Vacío/Escalón).	valid
ir_floor_front_right	digital	bool	boolean_value	IR de piso delantero derecho (true = Piso seguro, false = Vacío/Escalón).	valid
ir_floor_rear_left	digital	bool	boolean_value	IR de piso trasero izquierdo (true = Piso seguro, false = Vacío/Escalón).	valid
ir_floor_rear_right	digital	bool	boolean_value	IR de piso trasero derecho (true = Piso seguro, false = Vacío/Escalón).	valid
ir_lateral_left	digital	bool	boolean_value	IR lateral izquierdo en marco A04 (true = Despejado, false = Pared/Obstáculo).	valid
ir_lateral_right	digital	bool	boolean_value	IR lateral derecho en marco A04 (true = Despejado, false = Pared/Obstáculo).	valid

2.3 Lógica de los 6 Sensores Infrarrojos (4 de Piso y 2 Laterales)

El Rover V7.2 cuenta con 6 sensores infrarrojos divididos en dos funciones distintas: 4 sensores en las esquinas inferiores apuntando hacia el piso para evitar caídas por escaleras o desniveles, y 2 sensores en los costados del marco A04 apuntando lateralmente para detectar muros.

CONVENCIÓN OBLIGATORIA PARA LOS 6 SENSORES IR: TRUE = SEGURO / LIBRE

Físicamente, el comparador LM393 de un módulo IR entrega nivel bajo (LOW = 0) cuando detecta rebote de luz infrarroja. Sin embargo, el firmware del ESP32 normaliza la señal antes de construir el JSON para que tanto en piso como en laterales 'true' signifique siempre condición segura y 'false' signifique peligro u obstáculo.

Canales (channel_key)	Ubicación Física	Valor true (Seguro)	Valor false (Alerta)	Reacción del Robot	Qué mostrar en Web y Móvil
ir_floor_front_left **ir_floor_front_right**	Piso Frontal (Apuntando hacia abajo)	true = Hay piso firme (Rebote IR detectado)	false = VACÍO / ESCALÓN (Sin rebote IR)	Frena de inmediato, retrocede e invierte giro para no caer.	Indicador verde en true. Alerta roja 'Peligro de caída frontal' en false.
ir_floor_rear_left **ir_floor_rear_right**	Piso Trasero (Apuntando hacia abajo)	true = Hay piso firme (Rebote IR detectado)	false = VACÍO / ESCALÓN (Sin rebote IR)	Bloquea cualquier maniobra de reversa y avanza hacia zona segura.	Indicador verde en true. Alerta roja 'Peligro de caída trasera' en false.
ir_lateral_left **ir_lateral_right**	Laterales Marco A04 (Apuntando a los lados)	true = Costado libre (Sin pared cercana)	false = PARED / OBSTÁCULO (Rebote IR lateral)	Corrige rumbo suavemente para mantener distancia paralela al muro.	Indicador verde en true. Aviso ámbar 'Proximidad lateral' en false.

SECCIÓN 3

3. Ingesta Transaccional (ingest_telemetry) e Idempotencia

3.1 Diagrama y Secuencia Paso a Paso de la Función ingest_telemetry

La base de datos no recibe inserciones directas (INSERT) desde el exterior. Todo paquete entra exclusivamente a través de la función RPC public.ingest_telemetry(p_payload jsonb), la cual ejecuta toda la operación dentro de una única transacción atómica:

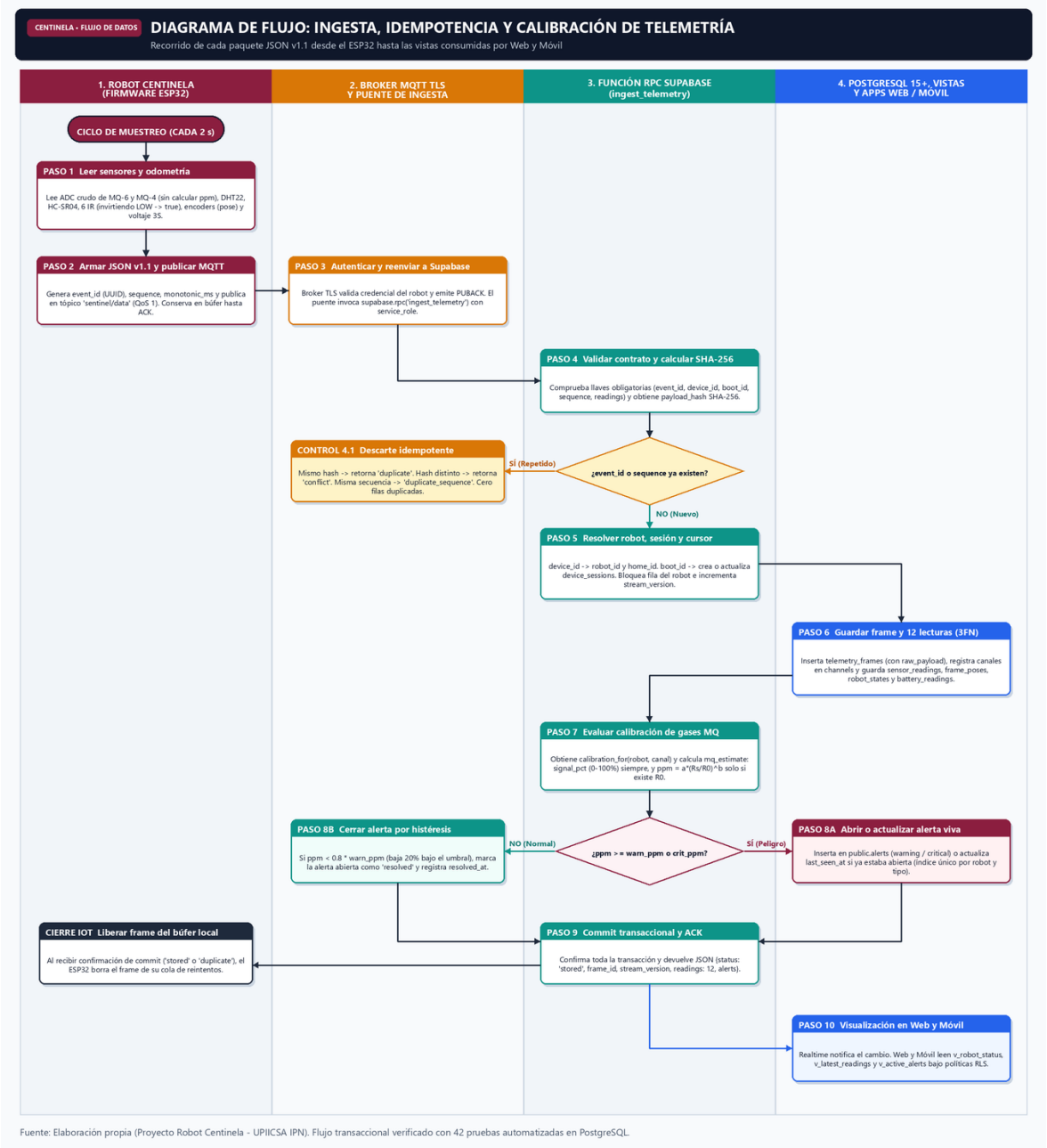


Figura 2: Diagrama de flujo transaccional en 4 carriles del procedimiento ingest_telemetry(jsonb).

- **Paso 1 (Validación y Hash):** Verifica que existan event_id, device_id, boot_id, sequence >= 0 y readings. Calcula el hash SHA-256 del payload completo.
- **Paso 2 (Idempotencia por event_id):** Busca si event_id ya existe en telemetry_frames. Si existe con el mismo hash, devuelve status = 'duplicate' sin duplicar registros; si el hash cambió, devuelve 'conflict'.
- **Paso 3 (Resolución de Dispositivo y Sesión):** Verifica que el device_id esté activo en devices, obtiene su robot_id, actualiza firmware_version y last_seen_at, y obtiene o crea la sesión en device_sessions para ese boot_id.
- **Paso 4 (Control de Secuencia y Cursor):** Comprueba que (device_session_id, sequence) no esté repetido e incrementa de forma atómica el contador robots.last_stream_version.
- **Paso 5 (Validación de Misión y Mapa):** Si el JSON incluye mission_id o map_version_id que aún no han sido dados de alta en la base de datos, los degrada limpiamente a NULL en lugar de abortar el paquete.
- **Paso 6 (Inserción en Cascada):** Inserta la cabecera en telemetry_frames y distribuye los datos en frame_poses, robot_states, battery_readings y sensor_readings.
- **Paso 7 (Evaluación de Alertas):** Invoca automáticamente evaluate_frame_alerts(frame_id) para abrir, actualizar o cerrar alertas de gas, abismo o batería baja.

3.2 Códigos de Respuesta del Servidor y Envío desde el Puente MQTT

La función ingest_telemetry devuelve siempre un objeto JSONB con el resultado de la operación. El puente MQTT debe actuar según el campo 'status':

status Devuelto	Campos Extra	Significado Técnico	Acción en el Puente MQTT / Backend
stored	frame_id, stream_version, readings, alerts	Paquete nuevo y válido. Se insertó en telemetry_frames y tablas hijas; incrementó stream_version.	Confirmar recepción (ACK) al broker MQTT.
duplicate	frame_id	El event_id ya existía con exactamente el mismo payload_hash (reintento de red legítimo).	Confirmar recepción (ACK) y descartar sin error.
conflict	frame_id	El event_id ya existía pero con un JSON distinto (colisión o alteración de datos).	Registrar en log de errores y no reintentar.
duplicate_sequence	sequence	Otro event_id ya utilizó ese número (boot_id, sequence) en la misma sesión de arranque.	Verificar contador sequence en firmware.
rejected	error, sqlstate	Faltan campos obligatorios, el device_id no existe/está inactivo, o un dato incumple restricciones.	Revisar el campo 'error' devuelto por la función.

Invocación de ingest_telemetry desde el Puente MQTT (Node.js / TypeScript)

```
// Ejemplo en Node.js (Puente MQTT -> Supabase) usando la llave service_role
import { createClient } from '@supabase/supabase-js';
const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_SERVICE_ROLE_KEY);

mqttClient.on('message', async (topic, messageBuffer) => {
  const payload = JSON.parse(messageBuffer.toString('utf-8'));
  const { data, error } = await supabase.rpc('ingest_telemetry', { p: payload });
```

Invocación de ingest_telemetry desde el Puente MQTT (Node.js / TypeScript) (continuación)

```
  if (error) {
    console.error('Error de red/SQL:', error.message);
    return; // Conservar en cola para reintento
  }
  if (data.status === 'stored' || data.status === 'duplicate') {
    console.log(`Frame procesado OK: ${data.status}, stream_version=${data.stream_version}`);
  } else {
    console.error(`Frame rechazado (${data.status}):`, data.error || data);
  }
});
```

SECCIÓN 4

4. Calibración de Gases MQ y Reglas Automáticas de Alerta

4.1 Fórmula de Conversión ADC a PPM y Parámetros por Sensor

Los sensores electroquímicos MQ-6 y MQ-4 forman un divisor resistivo con una resistencia de carga RL (10 kΩ). Las funciones SQL public.calibration_for(p_robot_id, p_channel_key), public.mq_estimate(p_raw, p) y public.gas_level(p_ppm, p) realizan la conversión en cuatro pasos:

- **1. Porcentaje de señal relativa (siempre disponible):** signal_pct = least(greatest((p_raw / 4095.0) * 100, 0), 100). Este valor se calcula siempre, incluso cuando el sensor aún no tiene calibrado su R0.
- **2. Conversión de ADC a voltaje y resistencia Rs:** V_s = (p_raw / 4095.0) * 3.3 V, y a continuación rs_kohm = RL * ((5.0 - V_s) / V_s) expresada en kΩ.
- **3. Ley de potencias cuando existe R0 calibrado:** Si la tabla gas_calibrations ya tiene un valor r0_kohm > 0 para ese robot y canal, calcula ratio = rs_kohm / r0_kohm y obtiene ppm = least(a * (ratio)^b, max_ppm).
- **4. Clasificación de nivel de riesgo (level):** Cuando existe ppm calculado, gas_level(ppm, params) compara contra warn_ppm y crit_ppm para devolver 'normal', 'warning' o 'critical'.

Canal	Gas Objetivo	Estado Inicial	r0_kohm Inicial	RL	Razón Aire Limpio	Constantes (a, b)	Umbral Warning	Umbral Critical
mq6_raw	LP (propano/butano)	default (sin R0)	null (hasta calibrar)	10.0 kΩ	10.0	a = 1009.2 b = -2.35	1000 ppm	2100 ppm
mq4_raw	Metano / gas natural	default (sin R0)	null (hasta calibrar)	10.0 kΩ	4.4	a = 1012.7 b = -2.786	2500 ppm	5000 ppm
mq7_raw	Monóxido de carbono (CO)	default (no instalado)	null	10.0 kΩ	27.5	a = 99.042 b = -1.518	35 ppm	100 ppm

4.2 Calibración de R0 en Aire Limpio y Ciclo de Alertas con Histéresis

Para pasar un sensor de 'uncalibrated' a calibrado (con cálculo real de ppm), se coloca el robot en un cuarto ventilado con aire limpio durante ~10 minutos y desde el backend (con llave service_role) se ejecuta select public.calibrate_r0_clean_air(p_robot_id, p_channel_key, p_window_minutes => 10, p_min_samples => 10). Esta función calcula la mediana de las lecturas ADC recientes (evitando que un pico aislado distorsione la muestra), obtiene Rs en aire limpio, lo divide entre clean_air_ratio (10.0 para MQ-6, 4.4 para MQ-4) y guarda automáticamente una nueva fila en gas_calibrations con status = 'field_clean_air', pasando confidence a 'medium'.

REGLA DE HISTÉRESIS DEL 80% EN ALERTAS DE GAS

Para evitar que una alerta se abra y se cierre intermitentemente cuando el gas oscila cerca del umbral, ingest_telemetry aplica una histéresis del 80%: una alerta abierta en public.alerts (con índice único parcial por robot y tipo) solo pasa a state = 'resolved' cuando la concentración estimada cae por debajo del 80% del umbral warning (ppm < 0.8 * warn_ppm).

SECCIÓN 5

5. Modelo Relacional en 3FN y Diccionario de Datos (16 Tablas)

5.1 Diagrama Entidad-Relación (DER) y Dominios del Esquema

El esquema de base de datos está compuesto por 16 tablas organizadas en 5 dominios funcionales bajo Tercera Forma Normal (3FN):

Dominio Funcional	Tablas que lo Integran	Propósito dentro del Sistema Centinela
1. Domicilios y Seguridad	homes, memberships	Aísla cada casa (multi-tenant) y define quién entra como 'reader' o 'technician'.
2. Robots y Sesiones	robots, devices, device_sessions	Vincula el chasis físico (CENTINELA-01) con su placa ESP32 y sus arranques (boot_id).
3. Mapas y Misiones	map_versions, missions	Guarda las versiones del plano 2D de la casa y los recorridos de patrullaje.
4. Telemetría Normalizada	telemetry_frames, channel_catalog, channels, sensor_readings, frame_poses, robot_states, battery_readings	Almacena la cabecera del paquete, las 12 lecturas por frame, la odometría 2D, el estado locomotor y la batería 3S sin columnas nulas redundantes.
5. Calibración y Alertas	gas_calibrations, alerts	Guarda las curvas químicas (R0, a, b) por robot y administra el ciclo de vida de alertas con histéresis.

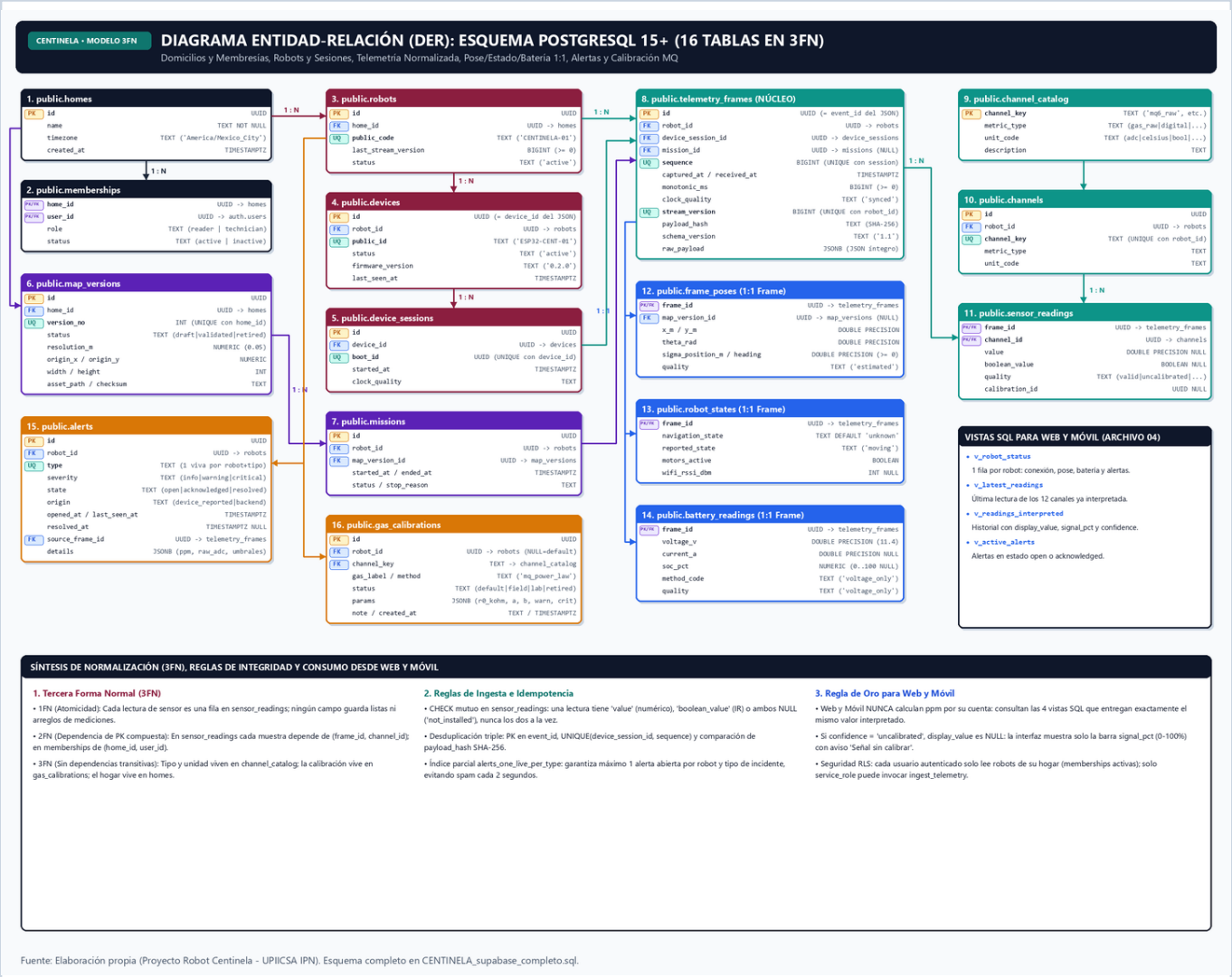


Figura 3: Diagrama Entidad-Relación (DER) completo con las 16 tablas en 3FN y las 4 vistas de consulta.

5.2 Diccionario de Datos de las 16 Tablas de PostgreSQL

A continuación se detalla la estructura columna por columna de las 16 tablas que conforman el esquema public:

Tabla 1: public.homes — Domicilios Registrados

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	ID único del domicilio.
name	TEXT	CHECK len > 0	NOT NULL	—	Nombre de la casa (ej. 'Casa Piloto Centinela').
timezone	TEXT	—	NOT NULL	'America/Mexico_City'	Zona horaria IANA para mostrar fechas locales.

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
created_at	TIMESTAMPTZ	—	NOT NULL	now()	Fecha de alta del domicilio.

Tabla 2: public.memberships — Usuarios y Roles por Domicilio

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
home_id	UUID	PK, FK -> homes	NOT NULL	—	Domicilio al que tiene acceso el usuario.
user_id	UUID	PK, FK -> auth.users	NOT NULL	—	Usuario autenticado en Supabase Auth.
role	TEXT	CHECK reader/tech	NOT NULL	—	'reader' (solo lectura) o 'technician' (acuse y calibración).
status	TEXT	CHECK active/inact	NOT NULL	—	Solo 'active' concede acceso en las políticas RLS.

Tabla 3: public.robots — Unidades Robot Centinela

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	ID interno del robot.
home_id	UUID	FK -> homes	NOT NULL	—	Domicilio donde opera el robot.
public_code	TEXT	UNIQUE	NOT NULL	—	Código visible en pantallas (ej. 'CENTINELA-01').
last_stream_version	BIGINT	CHECK >= 0	NOT NULL	0	Contador monotónico incrementado en cada frame almacenado.
status	TEXT	—	NOT NULL	'active'	Estado operativo del robot ('active', 'maintenance').

Tabla 4: public.devices — Placas Microcontroladoras (ESP32)

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	Corresponde al campo device_id del JSON.
robot_id	UUID	FK -> robots	NOT NULL	—	Robot donde está montado el ESP32.

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
public_id	TEXT	UNIQUE	NOT NULL	—	Etiqueta de hardware (ej. 'ESP32-CENT-01').
status	TEXT	—	NOT NULL	'active'	Si no es 'active', ingest_telemetry rechaza sus paquetes.
firmware_version	TEXT	—	NULL	NULL	Se actualiza automáticamente con cada JSON recibido.
last_seen_at	TIMESTAMPTZ	—	NULL	NULL	Hora del último paquete recibido (define online/stale/offline).

Tabla 5: public.device_sessions — Sesiones de Arranque (boot_id)

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	ID interno de la sesión de encendido.
device_id	UUID	FK -> devices	NOT NULL	—	ESP32 que inició la sesión.
boot_id	UUID	UNIQUE(device,boot)	NOT NULL	—	UUID generado por el ESP32 al encenderse o reiniciarse.
started_at	TIMESTAMPTZ	—	NOT NULL	now()	Hora del primer paquete recibido con ese boot_id.
clock_quality	TEXT	—	NOT NULL	'unknown'	Calidad del reloj reportada en la sesión.

Tabla 6: public.map_versions — Planos 2D del Domicilio

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	Corresponde a map_version_id del JSON.
home_id	UUID	FK -> homes	NOT NULL	—	Domicilio al que pertenece el plano.
version_no	INT	UNIQUE(home,ver)	NOT NULL	—	Número consecutivo de versión del mapa.
status	TEXT	CHECK draft/val/ret	NOT NULL	—	'draft', 'validated' o 'retired'.
resolution_m	NUMERIC	—	NOT NULL	0.05	Metros por píxel/celda del mapa (0.05 m = 5 cm).

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
origin_x / origin_y	NUMERIC	—	NOT NULL	0 / 0	Coordenadas del origen (0,0) en el plano.
width / height	INT	—	NOT NULL	—	Ancho y alto de la cuadrícula en celdas.
asset_path / checksum	TEXT	—	NOT NULL	—	Ruta de la imagen en Storage y hash de integridad.

Tabla 7: public.missions — Recorridos y Patrullajes

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	Corresponde a mission_id del JSON.
robot_id	UUID	FK -> robots	NOT NULL	—	Robot que ejecuta el recorrido.
map_version_id	UUID	FK -> map_versions	NOT NULL	—	Mapa utilizado en la misión.
started_at / ended_at	TIMESTAMPTZ	—	NOT NULL / NULL	now() / NULL	Inicio y fin del recorrido.
status / stop_reason	TEXT	—	NOT NULL / NULL	—	Estado ('running', 'completed') y motivo de parada.

Tabla 8: public.telemetry_frames — Cabecera de cada Paquete JSON

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	—	Es exactamente el event_id del JSON.
robot_id	UUID	FK -> robots	NOT NULL	—	Robot dueño del paquete.
device_session_id	UUID	FK -> device_sessions	NOT NULL	—	Sesión de arranque a la que pertenece.
mission_id	UUID	FK -> missions	NULL	NULL	Misión activa (o NULL si patrulla libremente).
sequence	BIGINT	UNIQUE(session,seq)	NOT NULL	—	Consecutivo del paquete dentro de la sesión.
captured_at / received_at	TIMESTAMPTZ	—	NULL / NOT NULL	— / now()	Hora del robot y hora real de llegada al servidor.

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
monotonic_ms	BIGINT	CHECK >= 0	NOT NULL	—	Milisegundos desde el arranque del ESP32.
clock_quality	TEXT	—	NOT NULL	—	'synced' o 'unsynced'.
stream_version	BIGINT	UNIQUE(robot,ver)	NOT NULL	—	Orden global de llegada por robot.
payload_hash	TEXT	—	NOT NULL	—	Hash SHA-256 del JSON para validar idempotencia.
schema_version	TEXT	—	NOT NULL	'1.1'	Versión del esquema JSON recibido.
raw_payload	JSONB	—	NULL	NULL	Respaldo íntegro del JSON crudo para auditoría.

Tabla 9: public.channel_catalog — Catálogo Maestro de los 12 Sensores

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
channel_key	TEXT	PRIMARY KEY	NOT NULL	—	Identificador fijo del canal (ej. 'mq6_raw', 'ir_lateral_left').
metric_type	TEXT	—	NOT NULL	—	Familia de medición ('gas_raw', 'temperature', 'digital'...).
unit_code	TEXT	—	NOT NULL	—	Unidad física ('adc', 'celsius', 'percent', 'meter', 'bool').
description	TEXT	—	NULL	NULL	Descripción del sensor físico y su función en el robot.

Tabla 10: public.channels — Canales Registrados por Robot

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	ID interno del canal para ese robot.
robot_id	UUID	FK -> robots	NOT NULL	—	Robot al que pertenece el canal.
channel_key	TEXT	FK -> channel_catalog	NOT NULL	—	Clave validada contra el catálogo maestro (UNIQUE con robot_id).
metric_type / unit_code	TEXT	—	NOT NULL	—	Copiados del catálogo maestro al crear el canal.

Tabla 11: public.sensor_readings — Lecturas de Sensores por Paquete

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
frame_id	UUID	PK, FK -> frames	NOT NULL	—	Paquete al que pertenece la lectura.
channel_id	UUID	PK, FK -> channels	NOT NULL	—	Canal del sensor medido.
value	DOUBLE PRECISION	CHECK mutuo	NULL	NULL	Valor numérico (para gases, temperatura, humedad y distancia).
boolean_value	BOOLEAN	CHECK mutuo	NULL	NULL	Valor booleano (exclusivo para los 6 sensores IR).
quality	TEXT	—	NOT NULL	—	'valid', 'uncalibrated', 'not_installed', 'warming', 'fault'.
calibration_id	UUID	—	NULL	NULL	Referencia opcional a la calibración activa.

Tabla 12: public.frame_poses — Posición y Orientación 2D (Encoders)

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
frame_id	UUID	PK, FK -> frames	NOT NULL	—	Relación 1:1 con telemetry_frames.
map_version_id	UUID	FK -> map_versions	NULL	NULL	Mapa de referencia (puede ser NULL si explora sin mapa previo).
x_m / y_m / theta_rad	DOUBLE PRECISION	—	NOT NULL	—	Posición X, Y en metros y ángulo theta en radianes.
sigma_position_m / heading	DOUBLE PRECISION	CHECK >= 0	NULL	NULL	Incertidumbre estimada de posición (m) y orientación (rad).
quality	TEXT	—	NOT NULL	—	Calidad de la pose (ej. 'estimated').

Tabla 13: public.robot_states — Estado Locomotor y Enlace

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
frame_id	UUID	PK, FK -> frames	NOT NULL	—	Relación 1:1 con telemetry_frames.
navigation_state	TEXT	—	NOT NULL	'unknown'	Estado de navegación ('autonomous', 'manual', 'unknown').
reported_state	TEXT	—	NOT NULL	—	Valor del campo robot_state del JSON ('moving', 'sampling'...).

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
motors_active	BOOLEAN	—	NULL	NULL	Estado de los motores en ese paquete.
wifi_rssi_dbm	INT	—	NULL	NULL	Potencia de señal WiFi en dBm (opcional).

Tabla 14: public.battery_readings — Medición de Batería 3S

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
frame_id	UUID	PK, FK -> frames	NOT NULL	—	Relación 1:1 con telemetry_frames.
voltage_v	DOUBLE PRECISION	CHECK >= 0	NULL	NULL	Voltaje del pack 3S (ej. 11.4 V).
current_a / soc_pct	DOUBLE / NUMERIC	— / 0..100	NULL	NULL	Corriente (A) y carga (%). En null cuando solo se mide voltaje.
method_code / quality	TEXT	—	NOT NULL	—	'voltage_only' en el hardware actual.

Tabla 15: public.alerts — Historial y Estado de Alertas

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	ID de la alerta.
robot_id	UUID	FK -> robots	NOT NULL	—	Robot que disparó la alerta.
type / severity	TEXT	— / CHECK	NOT NULL	—	Tipo ('gas_mq6', 'cliff_detected'...) y severidad ('info','warning','critical').
state / origin	TEXT	CHECK	NOT NULL	—	Estado ('open','acknowledged','resolved') y origen ('backend_rule').
opened_at / last_seen_at	TIMESTAMPTZ	—	NOT NULL	now()	Inicio y última confirmación de la condición de riesgo.
resolved_at	TIMESTAMPTZ	—	NULL	NULL	Se llena automáticamente cuando el riesgo desaparece.
source_frame_id / details	UUID / JSONB	FK -> frames	NULL	NULL	Último frame que actualizó la alerta y JSON con valores detallados.

Tabla 16: public.gas_calibrations — Parámetros Químicos MQ

Columna	Tipo SQL	Restricción / Clave	Nulabilidad	Default	Descripción
id	UUID	PRIMARY KEY	NOT NULL	gen_random_uuid()	ID del registro de calibración.
robot_id	UUID	FK -> robots	NULL	NULL	Robot calibrado (NULL = curva base global para todos los robots).
channel_key / gas_label	TEXT	FK -> catalog	NOT NULL	—	Canal ('mq6_raw', 'mq4_raw', 'mq7_raw') y nombre del gas.
method / status	TEXT	— / CHECK	NOT NULL	—	'mq_power_law' y estado ('default', 'field_clean_air', 'lab_validated').
params	JSONB	—	NOT NULL	—	Contiene adc_max, adc_vref, rl_kohm, r0_kohm, a, b, warn_ppm, crit_ppm.
note / created_at	TEXT / TIMESTAMPTZ	—	NULL / NOT NULL	— / now()	Nota de calibración y fecha de creación.

SECCIÓN 6

6. Manual Directo de Integración para los Equipos de Web y Móvil

6.1 Especificación de las 4 Vistas SQL Oficiales (Columnas Exactas)

Los desarrolladores de Web y Móvil no necesitan hacer JOINs manuales entre las 16 tablas. Toda la información lista para pintar en pantalla se obtiene consultando directamente estas 4 vistas:

1) Vista public.v_robot_status — Cabecera del Dashboard y Estado General

Columna Devuelta	Tipo de Dato	Qué contiene y cómo usarla en el Frontend
robot_id / public_code / home_id	UUID / TEXT / UUID	ID del robot, código visible ('CENTINELA-01') e ID del domicilio.
frame_id / captured_at / received_at	UUID / TIMESTAMPTZ	ID y marcas de tiempo del último paquete recibido de ese robot.
seconds_since_last / connection	NUMERIC / TEXT	Segundos desde el último paquete y estado: 'online' (<10 s), 'stale' (<60 s), 'offline' (>=60 s) o 'never'.
firmware_version	TEXT	Versión actual del firmware reportada por el ESP32 (ej. '0.2.0').
x_m / y_m / theta_rad / pose_quality	DOUBLE / TEXT	Última posición (X, Y) en metros, orientación en radianes y calidad ('estimated').
reported_state / navigation_state	TEXT	Estado operativo ('moving', 'sampling'...) y de navegación ('autonomous', 'unknown').
motors_active / wifi_rssi_dbm	BOOLEAN / INT	Indica si los motores están activos y la potencia WiFi en dBm.
battery_voltage_v / battery_soc_pct	DOUBLE / NUMERIC	Voltaje actual de la batería 3S (ej. 11.4 V) y porcentaje SOC (null si es voltage_only).
battery_quality	TEXT	Calidad/método de la lectura de batería ('voltage_only').
open_alerts / worst_severity	BIGINT / TEXT	Cantidad de alertas activas ('open' o 'acknowledged') y su severidad máxima ('info', 'warning', 'critical').

2) Vista public.v_latest_readings — Última Lectura de los 12 Sensores en Vivo

Columna Devuelta	Tipo de Dato	Qué contiene y cómo usarla en el Frontend
frame_id / robot_id / stream_version	UUID / UUID / BIGINT	ID del último frame del robot, ID del robot y versión del stream.
captured_at / received_at	TIMESTAMPTZ	Hora de captura en el robot y hora de recepción en la nube.
channel_key / metric_type	TEXT	Clave del canal ('mq6_raw', 'temperature', 'ir_floor_front_left'...) y tipo de métrica.

Columna Devuelta	Tipo de Dato	Qué contiene y cómo usarla en el Frontend
raw_value / boolean_value / raw_quality	DOUBLE / BOOL / TEXT	Valor crudo original (ADC, °C, %, m), valor booleano (sensores IR) y calidad reportada.
display_value / display_unit	DOUBLE / TEXT	Valor listo para mostrar en UI: para gases es ppm (o null si no hay R0); para otros sensores es su valor directo y unidad ('ppm', 'celsius', 'percent', 'meter', 'bool').
signal_pct	DOUBLE PRECISION	Solo para gases: porcentaje relativo del ADC (0 a 100%). Siempre disponible aunque display_value (ppm) sea null.
level	TEXT (Nullable)	Solo para gases calibrados: 'normal', 'warning' o 'critical' (null si aún no tiene R0).
confidence	TEXT	Confiabilidad del dato: 'high' (valid/lab), 'medium' (R0 aire limpio), 'uncalibrated' (sin R0) o 'not_installed'.
calibration_status / gas_label	TEXT (Nullable)	Estado de calibración ('default', 'field_clean_air', 'lab_validated') y nombre legible del gas ('LP (propano/butano)', 'Metano / gas natural').

3) Vista public.v_readings_interpreted — Historial y Series de Tiempo para Gráficas

Columna Devuelta	Tipo de Dato	Qué contiene y cómo usarla en el Frontend
frame_id / robot_id / stream_version	UUID / UUID / BIGINT	Identificadores del paquete para ordenar y filtrar series de tiempo.
captured_at / received_at	TIMESTAMPTZ	Eje temporal X para graficar la evolución histórica de cualquier canal.
channel_key / metric_type	TEXT	Canal a graficar (ej. filtrar channel_key = 'mq6_raw' o 'temperature').
raw_value / boolean_value / raw_quality	DOUBLE / BOOL / TEXT	Medición original guardada en sensor_readings.
display_value / display_unit	DOUBLE / TEXT	Valor ya convertido (ppm en gases calibrados; °C, % o m en sensores ambientales) y su unidad.
signal_pct / level / confidence	DOUBLE / TEXT / TEXT	Porcentaje de señal ADC (0–100%), nivel de alerta ('normal','warning','critical') y nivel de confianza.
calibration_status / gas_label	TEXT (Nullable)	Estado de la calibración empleada y etiqueta del gas.

4) Vista public.v_active_alerts — Bandeja de Alertas Abiertas y Reconocidas

Columna Devuelta	Tipo de Dato	Qué contiene y cómo usarla en el Frontend
id	UUID	ID de la alerta (se envía a rpc('ack_alert', { p_alert_id }) cuando el técnico la reconoce).
robot_id / public_code / home_id	UUID / TEXT / UUID	Identificación del robot ('CENTINELA-01') y domicilio donde ocurre la alerta.

Columna Devuelta	Tipo de Dato	Qué contiene y cómo usarla en el Frontend
type	TEXT	Tipo de incidente (ej. 'gas_mq6', 'gas_mq4').
severity	TEXT	Gravedad: 'info', 'warning' o 'critical'.
state / origin	TEXT	Estado actual ('open' o 'acknowledged') y origen ('backend_rule' o 'device_reported').
opened_at / last_seen_at	TIMESTAMPTZ	Cuándo inició la alerta y cuándo se confirmó por última vez.
details	JSONB	Objeto con {gas, ppm_estimated, raw_adc, calibration_status, warn_ppm, crit_ppm}.

6.2 Reglas de Interfaz (UI/UX): Cómo Mostrar Gases, Batería e IR

Para que la aplicación Web y la aplicación Móvil muestren exactamente los mismos datos y estados visuales, ambos equipos deben seguir esta tabla de comportamiento en interfaz:

Caso / Condición del Dato	Valores que recibe el Frontend	Qué debe pintar exactamente la Interfaz (Web y Móvil)
Gas con confidence = 'uncalibrated' (Estado inicial antes de calibrar R0)	display_value viene en null y signal_pct trae 0–100%.	Mostrar barra de 'Señal relativa: XX %' + etiqueta 'Sin calibrar R0 (solo señal relativa)'. PROHIBIDO mostrar '0 ppm'.
Gas con confidence = 'medium' o 'high' (Después de ejecutar calibrate_r0_clean_air)	display_value trae ppm reales (ej. 420.5) con display_unit = 'ppm' y level.	Mostrar el valor principal 'XX.X ppm' acompañado del semáforo según level ('normal' verde, 'warning' ámbar, 'critical' rojo).
Gas con confidence = 'not_installed' (Canal mq7_raw en fase actual)	raw_value = null, display_value = null, confidence = 'not_installed'.	Mostrar tarjeta en gris ('Sensor CO MQ-7: No instalado en esta versión'). No graficar ni mostrar alerta de fallo.
Batería con battery_quality = 'voltage_only' (Configuración actual del Rover V7.2)	battery_voltage_v = 11.4, battery_soc_pct = null.	Mostrar el voltaje exacto ('11.4 V — Batería 3S'). No mostrar '0%' cuando battery_soc_pct sea null.
Sensores IR de Piso y Laterales (6 canales booleanos)	boolean_value = true o false.	Pintar verde cuando sea true ('Piso firme' / 'Costado libre') y rojo/ámbar cuando sea false ('Abismo detectado' / 'Pared lateral').

6.3 Roles RLS y Ejemplos Listos de Código (Consultas, Realtime y RPC)

Al iniciar sesión con Supabase Auth, las políticas RLS filtran automáticamente las vistas según la tabla public.memberships. Los usuarios con rol 'reader' pueden consultar las 4 vistas y recibir eventos Realtime; los usuarios con rol 'technician' pueden además ejecutar ack_alert y calibrate_r0_clean_air.

Consultas, Realtime y RPC listos para copiar en Web y Móvil (@supabase/supabase-js)

```
// 1. Cabecera del Dashboard: estado del robot, batería, conexión y pose actual
const { data: robots } = await supabase.from('v_robot_status').select('*');

// 2. Tarjetas de los 12 sensores (incluye display_value, display_unit, signal_pct, level y confidence)
const { data: sensores } = await supabase
  .from('v_latest_readings')
  .select('*')
  .eq('robot_id', robotId);

// 3. Gráfica histórica de Gas LP (últimas 100 muestras de mq6_raw)
const { data: serieMq6 } = await supabase
  .from('v_readings_interpreted')
  .select('captured_at, raw_value, display_value, display_unit, signal_pct, level, confidence')
  .eq('robot_id', robotId)
  .eq('channel_key', 'mq6_raw')
  .order('captured_at', { ascending: false })
  .limit(100);

// 4. Bandeja de alertas activas ('open' y 'acknowledged')
const { data: alertas } = await supabase
  .from('v_active_alerts')
  .select('*')
  .eq('robot_id', robotId);

// 5. Suscripción en tiempo real (actualizar pantalla al llegar un frame o alerta nueva)
supabase.channel('centinela-live')
  .on('postgres_changes', {
    event: 'INSERT', schema: 'public', table: 'telemetry_frames', filter: `robot_id=eq.${robotId}`
  }, () => refrescarDashboard())
  .on('postgres_changes', {
    event: '*', schema: 'public', table: 'alerts', filter: `robot_id=eq.${robotId}`
  }, () => refrescarAlertas())
  .subscribe();

// 6. Reconocer una alerta activa (usuarios autenticados con rol 'technician')
await supabase.rpc('ack_alert', { p_alert_id: alertaId });
```

SECCIÓN 7

7. Despliegue en Supabase, Pruebas Verificadas y Código SQL

7.1 Orden de Ejecución en Supabase y Resultados del Test (42/42 OK)

En la carpeta del proyecto se entrega tanto el archivo unificado CENTINELA_supabase_completo.sql (listo para pegar y ejecutar en un solo paso en el SQL Editor de Supabase) como los 5 scripts modulares numerados:

- **1. sql/01_schema.sql:** Crea las 16 tablas en 3FN, las restricciones CHECK/UNIQUE, los índices de consulta y el catálogo maestro de los 12 canales.
- **2. sql/02_calibracion.sql:** Inserta las curvas base MQ-6, MQ-4 y MQ-7 y define interpret_gas_reading, calibrate_r0_clean_air, evaluate_frame_alerts y ack_alert.
- **3. sql/03_ingesta.sql:** Activa pgcrypto y crea la función transaccional idempotente ingest_telemetry(jsonb).
- **4. sql/04_vistas.sql:** Crea las 4 vistas oficiales con security_invoker = true para respetar RLS automáticamente.
- **5. sql/05_rls.sql:** Habilita Row Level Security en las 16 tablas y crea las funciones y políticas por rol ('reader' y 'technician').
- **6. sql/99_seed_demo.sql (Opcional para pruebas):** Crea la casa piloto, el robot CENTINELA-01 y el dispositivo ESP32 con los mismos UUIDs del archivo JSON_estructura.json.

VERIFICACIÓN AUTOMATIZADA EN POSTGRESQL: 42 / 42 PRUEBAS SUPERADAS (0 FALLOS)

El esquema completo y el JSON v1.1 fueron probados mediante el script pruebas/run_test.mjs sobre una instancia real de PostgreSQL (PGlite), verificando: 1) Ingesta del JSON_estructura.json (status='stored', 12 lecturas, pose y batería), 2) Idempotencia ante reenvío exacto (status='duplicate'), 3) Detección de conflicto ante alteración de payload (status='conflict'), 4) Disparo automático de alertas críticas por sensor de piso en false y gas elevado, 5) Calibración de R0 en aire limpio cambiando confidence de 'uncalibrated' a 'medium' con cálculo real de ppm, y 6) Cierre automático de alertas por histéresis al volver a valores normales.

7.2 Scripts SQL Modulares Completos (01_schema a 05_rls)

01_schema.sql — Tablas Base y Catálogo de 12 Canales (3FN)

```
-- ================================================================
-- CENTINELA - 01_schema.sql
-- Esquema base (version corregida para empatar con el JSON v1.1)
-- Correr en Supabase: SQL Editor. Orden: 01 -> 02 -> 03 -> 04
-- ================================================================
-- Nota: gen_random_uuid() ya viene en Postgres 13+, no hace falta uuid-ossp.

-- ---------------------------------------------------------------
-- 1. DOMICILIOS Y MEMBRESIAS
-- ---------------------------------------------------------------
create table if not exists public.homes (
    id uuid primary key default gen_random_uuid(),
    name text not null check (length(trim(name)) > 0),
    timezone text not null default 'America/Mexico_City',
    created_at timestamptz not null default now()
);

create table if not exists public.memberships (
```

01_schema.sql — Tablas Base y Catálogo de 12 Canales (3FN) (continuación)

```
    home_id uuid not null references public.homes(id) on delete restrict,
    user_id uuid not null references auth.users(id) on delete restrict,
    role text not null check (role in ('reader', 'technician')),
    status text not null check (status in ('active', 'inactive')),
    primary key (home_id, user_id)
);

-- ------------------------------------------------------------
-- 2. ROBOTS, DISPOSITIVOS Y SESIONES
-- ------------------------------------------------------------
create table if not exists public.robots (
    id uuid primary key default gen_random_uuid(),
    home_id uuid not null references public.homes(id) on delete restrict,
    public_code text not null unique,
    last_stream_version bigint not null default 0 check (last_stream_version >= 0),
    status text not null default 'active'
);

-- devices.id es el "device_id" que viaja en el JSON
create table if not exists public.devices (
    id uuid primary key default gen_random_uuid(),
    robot_id uuid not null references public.robots(id) on delete restrict,
    public_id text not null unique,
    status text not null default 'active',
    firmware_version text,
    last_seen_at timestamptz
);

create table if not exists public.device_sessions (
    id uuid primary key default gen_random_uuid(),
    device_id uuid not null references public.devices(id) on delete restrict,
    boot_id uuid not null,
    started_at timestamptz not null default now(),
    clock_quality text not null default 'unknown',
    unique (device_id, boot_id)
);

-- ------------------------------------------------------------
-- 3. MAPAS Y MISIONES
-- ------------------------------------------------------------
create table if not exists public.map_versions (
    id uuid primary key default gen_random_uuid(),
    home_id uuid not null references public.homes(id) on delete restrict,
    version_no int not null,
    status text not null check (status in ('draft', 'validated', 'retired')),
    resolution_m numeric not null default 0.05,
    origin_x numeric not null default 0,
    origin_y numeric not null default 0,
    width int not null,
    height int not null,
    asset_path text not null,
    checksum text not null,
    unique (home_id, version_no)
);

create table if not exists public.missions (
    id uuid primary key default gen_random_uuid(),
    robot_id uuid not null references public.robots(id) on delete restrict,
    map_version_id uuid not null references public.map_versions(id) on delete restrict,
    started_at timestamptz not null default now(),
```

01_schema.sql — Tablas Base y Catálogo de 12 Canales (3FN) (continuación)

```
    ended_at timestamptz,
    status text not null,
    stop_reason text
);

-- ------------------------------------------------------------------
-- 4. TELEMETRIA
-- ------------------------------------------------------------------
create table if not exists public.telemetry_frames (
    id uuid primary key,                        -- = event_id del JSON
    robot_id uuid not null references public.robots(id) on delete restrict,
    device_session_id uuid not null references public.device_sessions(id) on delete restrict,
    mission_id uuid references public.missions(id) on delete set null,
    sequence bigint not null check (sequence >= 0),
    captured_at timestamptz,
    received_at timestamptz not null default now(),
    monotonic_ms bigint not null check (monotonic_ms >= 0),
    clock_quality text not null,
    stream_version bigint not null,
    payload_hash text not null,
    schema_version text,                        -- NUEVO: version del JSON recibido
    raw_payload jsonb,                          -- NUEVO: JSON original, para auditoria
    unique (device_session_id, sequence),
    unique (robot_id, stream_version)
);

-- Catalogo de canales conocidos (NUEVO): define tipo y unidad de cada channel_key
create table if not exists public.channel_catalog (
    channel_key text primary key,
    metric_type text not null,
    unit_code text not null,
    description text
);

create table if not exists public.channels (
    id uuid primary key default gen_random_uuid(),
    robot_id uuid not null references public.robots(id) on delete restrict,
    channel_key text not null,
    metric_type text not null,
    unit_code text not null,
    unique (robot_id, channel_key)
);

create table if not exists public.sensor_readings (
    frame_id uuid not null references public.telemetry_frames(id) on delete cascade,
    channel_id uuid not null references public.channels(id) on delete restrict,
    value double precision,
    boolean_value boolean,
    quality text not null,
    calibration_id uuid,
    primary key (frame_id, channel_id),
    check (
        (value is not null and boolean_value is null) or
        (value is null and boolean_value is not null) or
        (value is null and boolean_value is null)
    )
);

-- CAMBIO: map_version_id ahora acepta null (el robot puede mandar pose sin mapa)
create table if not exists public.frame_poses (
```

01_schema.sql — Tablas Base y Catálogo de 12 Canales (3FN) (continuación)

```
    frame_id uuid primary key references public.telemetry_frames(id) on delete cascade,
    map_version_id uuid references public.map_versions(id) on delete restrict,
    x_m double precision not null,
    y_m double precision not null,
    theta_rad double precision not null,
    sigma_position_m double precision check (sigma_position_m >= 0),
    sigma_heading_rad double precision check (sigma_heading_rad >= 0),
    quality text not null
);

-- CAMBIO: navigation_state con default (el JSON solo manda "robot_state")
create table if not exists public.robot_states (
    frame_id uuid primary key references public.telemetry_frames(id) on delete cascade,
    navigation_state text not null default 'unknown',
    reported_state text not null,
    motors_active boolean,
    wifi_rssi_dbm int
);

create table if not exists public.battery_readings (
    frame_id uuid primary key references public.telemetry_frames(id) on delete cascade,
    voltage_v double precision check (voltage_v >= 0),
    current_a double precision,
    soc_pct numeric check (soc_pct >= 0 and soc_pct <= 100),
    method_code text not null,
    quality text not null
);

-- ---------------------------------------------------------------
-- 5. ALERTAS
-- ---------------------------------------------------------------
create table if not exists public.alerts (
    id uuid primary key default gen_random_uuid(),
    robot_id uuid not null references public.robots(id) on delete restrict,
    type text not null,
    severity text not null check (severity in ('info', 'warning', 'critical')),
    state text not null check (state in ('open', 'acknowledged', 'resolved')),
    origin text not null check (origin in ('device_reported', 'backend_rule')),
    opened_at timestamptz not null default now(),
    last_seen_at timestamptz not null default now(),
    resolved_at timestamptz,
    source_frame_id uuid references public.telemetry_frames(id) on delete set null, -- NUEVO
    details jsonb                                                                   -- NUEVO
);

-- Una sola alerta viva por robot y tipo (evita spam de alertas repetidas)
create unique index if not exists alerts_one_live_per_type
    on public.alerts(robot_id, type) where state in ('open', 'acknowledged');

-- ---------------------------------------------------------------
-- 6. INDICES
-- ---------------------------------------------------------------
create index if not exists frames_capture_idx on public.telemetry_frames(robot_id, captured_at desc);
create index if not exists frames_received_idx on public.telemetry_frames(robot_id, received_at desc);
create index if not exists readings_channel_idx on public.sensor_readings(channel_id, frame_id);
create index if not exists alerts_robot_state_idx on public.alerts(robot_id, state);

-- ---------------------------------------------------------------
-- 7. CATALOGO INICIAL DE CANALES (los que manda el firmware 0.2.0)
-- ---------------------------------------------------------------
```

01_schema.sql — Tablas Base y Catálogo de 12 Canales (3FN) (continuación)

```
insert into public.channel_catalog (channel_key, metric_type, unit_code, description) values
    ('mq6_raw',              'gas_raw',   'adc',     'MQ-6 (LP/propano-butano), lectura ADC cruda'),
    ('mq4_raw',              'gas_raw',   'adc',     'MQ-4 (metano/gas natural), lectura ADC cruda'),
    ('mq7_raw',              'gas_raw',   'adc',     'MQ-7 (monoxido de carbono), lectura ADC cruda'),
    ('temperature',          'temperature','celsius','DHT22 temperatura'),
    ('humidity',             'humidity',  'percent', 'DHT22 humedad relativa'),
    ('front_distance_m',     'distance',  'meter',   'HC-SR04 distancia frontal'),
    ('ir_floor_front_left',  'digital',   'bool',    'IR piso frente izquierdo'),
    ('ir_floor_front_right', 'digital',   'bool',    'IR piso frente derecho'),
    ('ir_floor_rear_left',   'digital',   'bool',    'IR piso atras izquierdo'),
    ('ir_floor_rear_right',  'digital',   'bool',    'IR piso atras derecho'),
    ('ir_lateral_left',      'digital',   'bool',    'IR lateral izquierdo (pared)'),
    ('ir_lateral_right',     'digital',   'bool',    'IR lateral derecho (pared)')
on conflict (channel_key) do nothing;
```

02_calibracion.sql — Curvas MQ, Conversión a PPM y Reglas de Alerta

```
-- ================================================================
-- CENTINELA - 02_calibracion.sql
-- Interpretacion de gases (MQ) en el BACKEND. El robot solo manda ADC crudo.
-- Web y movil NO calculan nada: consultan las vistas del archivo 04.
-- ================================================================

-- Calibraciones versionadas. robot_id NULL = valores por defecto para cualquier robot.
create table if not exists public.gas_calibrations (
    id uuid primary key default gen_random_uuid(),
    robot_id uuid references public.robots(id) on delete restrict,
    channel_key text not null references public.channel_catalog(channel_key),
    gas_label text not null,
    method text not null default 'mq_power_law',
    -- default         = curva de referencia, SIN R0 (solo muestra senal relativa)
    -- field_clean_air = R0 medido en aire limpio con el propio robot (estimacion)
    -- lab_validated   = calibrado contra gas patron (valor confiable)
    status text not null check (status in ('default', 'field_clean_air', 'lab_validated', 'retired')),
    params jsonb not null,
    note text,
    created_at timestamptz not null default now()
);
create index if not exists gas_cal_lookup_idx on public.gas_calibrations(channel_key, robot_id, status);

-- ----------------------------------------------------------------
-- Parametros (jsonb):
--   adc_max        resolucion del ADC (ESP32 = 4095)
--   adc_vref       voltaje de referencia del ADC (ESP32 ~ 3.3)
--   divider_ratio  factor del divisor de voltaje del pin AO (1.0 si va directo)
--   vcc            alimentacion del sensor (5.0)
--   rl_kohm        resistencia de carga del modulo (10 en la mayoria)
--   r0_kohm        Rs en aire limpio / clean_air_ratio (null = sin calibrar)
--   clean_air_ratio  Rs/R0 en aire limpio segun datasheet
--   a, b           curva ppm = a * (Rs/R0)^b
--   max_ppm        tope del rango del sensor
--   warn_ppm, crit_ppm  umbrales de alerta (propuesta: 5% y 10% del LIE)
-- OJO: adc_vref, divider_ratio y rl_kohm deben coincidir con el circuito real.
-- ----------------------------------------------------------------
insert into public.gas_calibrations (robot_id, channel_key, gas_label, status, params, note)
select null, v.channel_key, v.gas_label, 'default', v.params::jsonb, v.note
from (values
    ('mq6_raw', 'LP (propano/butano)',
     '{"adc_max":4095,"adc_vref":3.3,"divider_ratio":1.0,"vcc":5.0,"rl_kohm":10,"r0_kohm":null,
      "clean_air_ratio":10,"a":1009.2,"b":-2.35,"max_ppm":10000,"warn_ppm":1000,"crit_ppm":2100}',
```

02_calibracion.sql — Curvas MQ, Conversión a PPM y Reglas de Alerta (continuación)

```
     'Curva de referencia (libreria MQUnifiedsensor). Umbrales: ~5% y 10% del LIE del propano.'),
    ('mq4_raw', 'Metano / gas natural',
     '{"adc_max":4095,"adc_vref":3.3,"divider_ratio":1.0,"vcc":5.0,"rl_kohm":10,"r0_kohm":null,
      "clean_air_ratio":4.4,"a":1012.7,"b":-2.786,"max_ppm":10000,"warn_ppm":2500,"crit_ppm":5000}',
     'Curva de referencia (libreria MQUnifiedsensor). Umbrales: ~5% y 10% del LIE del metano.'),
    ('mq7_raw', 'Monoxido de carbono',
     '{"adc_max":4095,"adc_vref":3.3,"divider_ratio":1.0,"vcc":5.0,"rl_kohm":10,"r0_kohm":null,
      "clean_air_ratio":27.5,"a":99.042,"b":-1.518,"max_ppm":2000,"warn_ppm":35,"crit_ppm":100}',
     'MQ-7 necesita ciclo de calentado (5V/1.4V); sin ese ciclo la estimacion no es confiable.')
) as v(channel_key, gas_label, params, note)
where not exists (
    select 1 from public.gas_calibrations g
    where g.robot_id is null and g.channel_key = v.channel_key and g.status = 'default'
);

-- ---------------------------------------------------------------------
-- Calibracion elegida para un robot + canal (la mas confiable y reciente)
-- ---------------------------------------------------------------------
create or replace function public.calibration_for(p_robot_id uuid, p_channel_key text)
returns public.gas_calibrations
language sql stable
set search_path = public
as $$
    select g.*
    from public.gas_calibrations g
    where g.channel_key = p_channel_key
      and g.status <> 'retired'
      and (g.robot_id = p_robot_id or g.robot_id is null)
    order by (g.robot_id is not null) desc,
             case g.status when 'lab_validated' then 3 when 'field_clean_air' then 2 else 1 end desc,
             g.created_at desc
    limit 1;
$$;

-- ---------------------------------------------------------------------
-- Estimacion MQ: ADC crudo -> senal relativa, Rs y ppm estimados
-- ppm solo sale si hay R0 (calibracion). Sin R0 solo hay senal relativa.
-- ---------------------------------------------------------------------
create or replace function public.mq_estimate(p_raw double precision, p jsonb)
returns table (signal_pct double precision, rs_kohm double precision, ratio double precision, ppm double
      precision)
language plpgsql immutable
as $$
declare
    v_adc_max double precision := coalesce((p->>'adc_max')::double precision, 4095);
    v_vref    double precision := coalesce((p->>'adc_vref')::double precision, 3.3);
    v_div     double precision := coalesce((p->>'divider_ratio')::double precision, 1.0);
    v_vcc     double precision := coalesce((p->>'vcc')::double precision, 5.0);
    v_rl      double precision := coalesce((p->>'rl_kohm')::double precision, 10);
    v_r0      double precision := (p->>'r0_kohm')::double precision;
    v_a       double precision := (p->>'a')::double precision;
    v_b       double precision := (p->>'b')::double precision;
    v_max     double precision := coalesce((p->>'max_ppm')::double precision, 10000);
    v_vs      double precision;
begin
    signal_pct := null; rs_kohm := null; ratio := null; ppm := null;
    if p_raw is not null then
        signal_pct := least(greatest(p_raw / v_adc_max * 100, 0), 100);
        v_vs := p_raw / v_adc_max * v_vref * v_div;
        if v_vs > 0.001 and v_vs < v_vcc then
```

02_calibracion.sql — Curvas MQ, Conversión a PPM y Reglas de Alerta (continuación)

```
            rs_kohm := v_rl * (v_vcc - v_vs) / v_vs;
            if v_r0 is not null and v_r0 > 0 and v_a is not null and v_b is not null then
                ratio := rs_kohm / v_r0;
                ppm := least(v_a * power(ratio, v_b), v_max);
            end if;
        end if;
    end if;
    return next;
end;
$$;

-- ---------------------------------------------------------------------
-- Nivel de riesgo a partir de ppm estimados y umbrales
-- ---------------------------------------------------------------------
create or replace function public.gas_level(p_ppm double precision, p jsonb)
returns text
language sql immutable
as $$
    select case
        when p_ppm is null then null
        when p_ppm >= (p->>'crit_ppm')::double precision then 'critical'
        when p_ppm >= (p->>'warn_ppm')::double precision then 'warning'
        else 'normal'
    end;
$$;

-- ---------------------------------------------------------------------
-- Calibracion en aire limpio: el robot se deja ~10 min en un cuarto ventilado
-- y esta funcion fija R0 a partir de las lecturas recientes.
-- Uso:  select public.calibrate_r0_clean_air('<robot_id>', 'mq6_raw');
-- ---------------------------------------------------------------------
create or replace function public.calibrate_r0_clean_air(
    p_robot_id uuid,
    p_channel_key text,
    p_window_minutes int default 10,
    p_min_samples int default 10,
    p_note text default null
) returns uuid
language plpgsql
set search_path = public
as $$
declare
    v_base public.gas_calibrations;
    v_avg double precision;
    v_n bigint;
    v_rs double precision;
    v_ratio double precision;
    v_r0 double precision;
    v_new uuid;
begin
    v_base := public.calibration_for(p_robot_id, p_channel_key);
    if v_base.id is null then
        raise exception 'No hay calibracion base para el canal %', p_channel_key;
    end if;

    -- mediana (no promedio): un picon de gas durante la calibracion no la arruina
    select percentile_cont(0.5) within group (order by sr.value), count(*) into v_avg, v_n
    from public.sensor_readings sr
    join public.channels c on c.id = sr.channel_id
    join public.telemetry_frames f on f.id = sr.frame_id
```

02_calibracion.sql — Curvas MQ, Conversión a PPM y Reglas de Alerta (continuación)

```
    where c.robot_id = p_robot_id
      and c.channel_key = p_channel_key
      and sr.value is not null
      and sr.quality in ('uncalibrated', 'valid')
      and f.received_at >= now() - make_interval(mins => p_window_minutes);

    if v_n < p_min_samples then
        raise exception 'Muestras insuficientes para %: % de % requeridas', p_channel_key, v_n, p_min_samples;
    end if;

    select e.rs_kohm into v_rs from public.mq_estimate(v_avg, v_base.params) e;
    v_ratio := (v_base.params->>'clean_air_ratio')::double precision;
    if v_rs is null or v_ratio is null or v_ratio <= 0 then
        raise exception 'No se pudo calcular Rs (revisar adc_vref/divider_ratio). Promedio ADC=%', v_avg;
    end if;
    v_r0 := v_rs / v_ratio;

    update public.gas_calibrations
       set status = 'retired'
     where robot_id = p_robot_id and channel_key = p_channel_key and status = 'field_clean_air';

    insert into public.gas_calibrations (robot_id, channel_key, gas_label, status, params, note)
    values (p_robot_id, p_channel_key, v_base.gas_label, 'field_clean_air',
            v_base.params || jsonb_build_object('r0_kohm', v_r0),
            coalesce(p_note, format('R0 en aire limpio: %s muestras, ADC mediana %s', v_n, round(v_avg::numeric,
      1))))
    returning id into v_new;
    return v_new;
end;
$$;
```

03_ingesta.sql — RPC Transaccional Idempotente ingest_telemetry(jsonb)

```
-- ====================================================================
-- CENTINELA - 03_ingesta.sql
-- Punto unico de entrada de telemetria:  select public.ingest_telemetry('<json>'::jsonb);
-- Desde supabase-js:  supabase.rpc('ingest_telemetry', { p: payload })   (con service_role)
-- Hace lo que el ESP32 NO debe hacer: resolver robot, sesion, stream_version,
-- hash, idempotencia, alta de canales, pose, estado, bateria y alertas de gas.
-- ====================================================================
create or replace function public.ingest_telemetry(p jsonb)
returns jsonb
language plpgsql
security definer
set search_path = public
as $$
declare
    v_event      uuid;
    v_device     uuid;
    v_robot      uuid;
    v_home       uuid;
    v_session    uuid;
    v_hash       text;
    v_old_hash   text;
    v_stream     bigint;
    v_mission    uuid;
    v_map        uuid;
    v_n_reads    int;
    v_alerts     jsonb := '[]'::jsonb;
    r            record;
```

03_ingesta.sql — RPC Transaccional Idempotente ingest_telemetry(jsonb) (continuación)

```
    v_cal        public.gas_calibrations;
    v_est        record;
    v_level      text;
    v_type       text;
begin
    -- 1. Validacion minima del contrato
    if p is null or jsonb_typeof(p) <> 'object' then
        raise exception 'payload no es un objeto JSON';
    end if;
    if not (p ? 'event_id' and p ? 'device_id' and p ? 'boot_id' and p ? 'sequence'
            and p ? 'monotonic_ms' and p ? 'clock_quality' and p ? 'readings') then
        raise exception 'faltan campos obligatorios (event_id, device_id, boot_id, sequence, monotonic_ms,
      clock_quality, readings)';
    end if;
    if jsonb_typeof(p->'readings') <> 'array' then
        raise exception 'readings debe ser un arreglo';
    end if;

    v_event := (p->>'event_id')::uuid;
    v_hash  := encode(sha256(convert_to(p::text, 'UTF8')), 'hex');

    -- 2. Idempotencia: el mismo event_id no se guarda dos veces
    select payload_hash into v_old_hash from public.telemetry_frames where id = v_event;
    if found then
        return jsonb_build_object(
            'status', case when v_old_hash = v_hash then 'duplicate' else 'conflict' end,
            'frame_id', v_event);
    end if;

    -- 3. device_id (JSON) -> robot_id
    select d.id, d.robot_id, rb.home_id into v_device, v_robot, v_home
    from public.devices d
    join public.robots rb on rb.id = d.robot_id
    where d.id = (p->>'device_id')::uuid and d.status = 'active';
    if v_device is null then
        raise exception 'device_id desconocido o inactivo: %', p->>'device_id';
    end if;

    -- 4. boot_id -> sesion del dispositivo (se crea si el robot acaba de reiniciar)
    insert into public.device_sessions (device_id, boot_id, clock_quality)
    values (v_device, (p->>'boot_id')::uuid, p->>'clock_quality')
    on conflict (device_id, boot_id) do update set clock_quality = excluded.clock_quality
    returning id into v_session;

    if exists (select 1 from public.telemetry_frames
               where device_session_id = v_session and sequence = (p->>'sequence')::bigint) then
        return jsonb_build_object('status', 'duplicate_sequence',
                                  'sequence', (p->>'sequence')::bigint);
    end if;

    -- 5. Referencias opcionales: si la mision/mapa no existen en la nube se guarda null
    select m.id into v_mission from public.missions m
     where m.id = nullif(p->>'mission_id', '')::uuid and m.robot_id = v_robot;
    select mv.id into v_map from public.map_versions mv
     where mv.id = nullif(p->>'map_version_id', '')::uuid and mv.home_id = v_home;

    -- 6. stream_version consecutivo por robot (lo asigna la nube, no el robot)
    update public.robots set last_stream_version = last_stream_version + 1
     where id = v_robot returning last_stream_version into v_stream;
```

03_ingesta.sql — RPC Transaccional Idempotente ingest_telemetry(jsonb) (continuación)

```
    -- 7. Frame
    insert into public.telemetry_frames
        (id, robot_id, device_session_id, mission_id, sequence, captured_at, monotonic_ms,
         clock_quality, stream_version, payload_hash, schema_version, raw_payload)
    values
        (v_event, v_robot, v_session, v_mission, (p->>'sequence')::bigint,
         nullif(p->>'captured_at', '')::timestamptz, (p->>'monotonic_ms')::bigint,
         p->>'clock_quality', v_stream, v_hash, p->>'schema_version', p);

    -- 8. Lecturas (canales nuevos se dan de alta solos)
    insert into public.channels (robot_id, channel_key, metric_type, unit_code)
    select distinct v_robot, x->>'channel_key',
           coalesce(cc.metric_type, 'unknown'), coalesce(cc.unit_code, 'unknown')
    from jsonb_array_elements(p->'readings') x
    left join public.channel_catalog cc on cc.channel_key = x->>'channel_key'
    on conflict (robot_id, channel_key) do nothing;

    insert into public.sensor_readings (frame_id, channel_id, value, boolean_value, quality, calibration_id)
    select v_event, c.id,
           (x->>'value')::double precision,
           (x->>'boolean_value')::boolean,
           coalesce(x->>'quality', 'unknown'),
           nullif(x->>'calibration_id', '')::uuid
    from jsonb_array_elements(p->'readings') x
    join public.channels c on c.robot_id = v_robot and c.channel_key = x->>'channel_key';
    get diagnostics v_n_reads = row_count;

    -- 9. Pose
    if jsonb_typeof(p->'pose') = 'object' then
        insert into public.frame_poses
            (frame_id, map_version_id, x_m, y_m, theta_rad, sigma_position_m, sigma_heading_rad, quality)
        values
            (v_event, v_map,
             (p->'pose'->>'x_m')::double precision,
             (p->'pose'->>'y_m')::double precision,
             (p->'pose'->>'theta_rad')::double precision,
             (p->'pose'->>'sigma_position_m')::double precision,
             (p->'pose'->>'sigma_heading_rad')::double precision,
             coalesce(p->'pose'->>'quality', 'unknown'));
    end if;

    -- 10. Estado del robot
    insert into public.robot_states (frame_id, navigation_state, reported_state, motors_active, wifi_rssi_dbm)
    values (v_event,
            coalesce(p->>'navigation_state', 'unknown'),
            coalesce(p->>'robot_state', 'unknown'),
            (p->>'motors_active')::boolean,
            (p->>'wifi_rssi_dbm')::int);

    -- 11. Bateria
    if jsonb_typeof(p->'battery') = 'object' then
        insert into public.battery_readings (frame_id, voltage_v, current_a, soc_pct, method_code, quality)
        values (v_event,
                (p->'battery'->>'voltage_v')::double precision,
                (p->'battery'->>'current_a')::double precision,
                (p->'battery'->>'soc_pct')::numeric,
                coalesce(p->'battery'->>'method_code', p->'battery'->>'quality', 'unknown'),
                coalesce(p->'battery'->>'quality', 'unknown'));
    end if;
```

03_ingesta.sql — RPC Transaccional Idempotente ingest_telemetry(jsonb) (continuación)

```
    -- 12. Dispositivo vivo
    update public.devices
       set last_seen_at = now(),
           firmware_version = coalesce(p->>'firmware_version', firmware_version)
     where id = v_device;

    -- 13. Reglas de gas en backend (solo si hay calibracion con R0 -> ppm estimados)
    for r in
        select c.channel_key, sr.value
        from public.sensor_readings sr
        join public.channels c on c.id = sr.channel_id
        where sr.frame_id = v_event and c.metric_type = 'gas_raw'
          and sr.value is not null and sr.quality in ('uncalibrated', 'valid')
    loop
        v_cal := public.calibration_for(v_robot, r.channel_key);
        continue when v_cal.id is null;
        select * into v_est from public.mq_estimate(r.value, v_cal.params);
        continue when v_est.ppm is null;
        v_level := public.gas_level(v_est.ppm, v_cal.params);
        v_type  := 'gas_' || replace(r.channel_key, '_raw', '');

        if v_level in ('warning', 'critical') then
            insert into public.alerts (robot_id, type, severity, state, origin, source_frame_id, details)
            values (v_robot, v_type, v_level, 'open', 'backend_rule', v_event,
                    jsonb_build_object('gas', v_cal.gas_label, 'ppm_estimated', round(v_est.ppm::numeric, 1),
                                       'raw_adc', r.value, 'calibration_status', v_cal.status,
                                       'warn_ppm', (v_cal.params->>'warn_ppm')::numeric,
                                       'crit_ppm', (v_cal.params->>'crit_ppm')::numeric))
            on conflict (robot_id, type) where state in ('open', 'acknowledged')
            do update set severity = excluded.severity, last_seen_at = now(),
                          source_frame_id = excluded.source_frame_id, details = excluded.details;
            v_alerts := v_alerts || jsonb_build_object('type', v_type, 'severity', v_level);
        elsif v_est.ppm < 0.8 * (v_cal.params->>'warn_ppm')::double precision then
            -- histeresis: se cierra la alerta solo cuando baja bien por debajo del umbral
            update public.alerts set state = 'resolved', resolved_at = now()
             where robot_id = v_robot and type = v_type and state in ('open', 'acknowledged');
        end if;
    end loop;

    return jsonb_build_object('status', 'stored', 'frame_id', v_event,
                              'stream_version', v_stream, 'readings', v_n_reads,
                              'alerts', v_alerts);

exception when others then
    return jsonb_build_object('status', 'rejected', 'error', sqlerrm, 'sqlstate', sqlstate);
end;
$$;

revoke all on function public.ingest_telemetry(jsonb) from public, anon, authenticated;
grant execute on function public.ingest_telemetry(jsonb) to service_role;

revoke all on function public.calibrate_r0_clean_air(uuid, text, int, int, text) from public, anon, authenticated;
grant execute on function public.calibrate_r0_clean_air(uuid, text, int, int, text) to service_role;
```

04_vistas.sql — las 4 Vistas Oficiales para Equipos Web y Móvil

```
-- ================================================================
-- CENTINELA - 04_vistas.sql
-- Vistas para WEB y MOVIL. Ambas apps consultan EXACTAMENTE lo mismo, asi
-- nunca habra "80 ppm" en la web y "950 ppm" en la app.
```

04_vistas.sql — las 4 Vistas Oficiales para Equipos Web y Móvil (continuación)

```
-- security_invoker = true -> respeta RLS del usuario que consulta.
-- ====================================================================

-- Todas las lecturas ya interpretadas
create or replace view public.v_readings_interpreted
with (security_invoker = true) as
select
    f.id              as frame_id,
    f.robot_id,
    f.stream_version,
    f.captured_at,
    f.received_at,
    c.channel_key,
    c.metric_type,
    sr.value          as raw_value,
    sr.boolean_value,
    sr.quality        as raw_quality,
    -- valor listo para mostrar: gas = ppm estimados (null si no hay calibracion)
    case when c.metric_type = 'gas_raw' then round(est.ppm::numeric, 1)::double precision
         else sr.value end                                  as display_value,
    case when c.metric_type = 'gas_raw' then 'ppm' else c.unit_code end as display_unit,
    -- senal relativa 0-100 (siempre disponible para gases, sirve de barra/tacometro)
    case when c.metric_type = 'gas_raw' then round(est.signal_pct::numeric, 1)::double precision end as
      signal_pct,
    case when c.metric_type = 'gas_raw' then public.gas_level(est.ppm, cal.params) end as level,
    case
        when sr.quality = 'not_installed' then 'not_installed'
        when c.metric_type = 'gas_raw' then
            case cal.status when 'lab_validated'   then 'high'
                           when 'field_clean_air' then 'medium'
                           else 'uncalibrated' end
        when sr.quality = 'valid' then 'high'
        else sr.quality
    end                                                    as confidence,
    case when c.metric_type = 'gas_raw' then cal.status end   as calibration_status,
    case when c.metric_type = 'gas_raw' then cal.gas_label end as gas_label
from public.sensor_readings sr
join public.telemetry_frames f on f.id = sr.frame_id
join public.channels c         on c.id = sr.channel_id
left join lateral public.calibration_for(f.robot_id, c.channel_key) cal
       on c.metric_type = 'gas_raw'
left join lateral public.mq_estimate(sr.value, cal.params) est
       on c.metric_type = 'gas_raw' and sr.value is not null and cal.params is not null;

comment on view public.v_readings_interpreted is
'confidence: high | medium (R0 en aire limpio, estimacion) | uncalibrated (mostrar solo signal_pct, NO ppm) |
      not_installed';

-- Ultima lectura de cada canal por robot (para el dashboard en vivo)
create or replace view public.v_latest_readings
with (security_invoker = true) as
select v.*
from public.v_readings_interpreted v
where v.frame_id in (
    select distinct on (robot_id) id
    from public.telemetry_frames
    order by robot_id, stream_version desc
);

-- Estado actual de cada robot (una fila por robot)
```

04_vistas.sql — las 4 Vistas Oficiales para Equipos Web y Móvil (continuación)

```
create or replace view public.v_robot_status
with (security_invoker = true) as
select
    rb.id                       as robot_id,
    rb.public_code,
    rb.home_id,
    lf.id                       as frame_id,
    lf.captured_at,
    lf.received_at,
    round(extract(epoch from (now() - lf.received_at))::numeric, 1) as seconds_since_last,
    case
        when lf.id is null then 'never'
        when now() - lf.received_at < interval '10 seconds' then 'online'
        when now() - lf.received_at < interval '60 seconds' then 'stale'
        else 'offline'
    end                         as connection,
    d.firmware_version,
    fp.x_m, fp.y_m, fp.theta_rad, fp.quality as pose_quality,
    rs.reported_state, rs.navigation_state, rs.motors_active, rs.wifi_rssi_dbm,
    br.voltage_v                as battery_voltage_v,
    br.soc_pct                  as battery_soc_pct,
    br.quality                  as battery_quality,
    coalesce(al.open_alerts, 0) as open_alerts,
    al.worst_severity
from public.robots rb
left join lateral (
    select * from public.telemetry_frames t
    where t.robot_id = rb.id order by t.stream_version desc limit 1
) lf on true
left join public.devices d         on d.robot_id = rb.id
left join public.frame_poses fp    on fp.frame_id = lf.id
left join public.robot_states rs   on rs.frame_id = lf.id
left join public.battery_readings br on br.frame_id = lf.id
left join lateral (
    select count(*) as open_alerts,
           case when bool_or(severity = 'critical') then 'critical'
                when bool_or(severity = 'warning')  then 'warning'
                else 'info' end as worst_severity
    from public.alerts a
    where a.robot_id = rb.id and a.state in ('open', 'acknowledged')
) al on true;

-- Alertas vivas con datos legibles
create or replace view public.v_active_alerts
with (security_invoker = true) as
select a.id, a.robot_id, rb.public_code, rb.home_id, a.type, a.severity, a.state, a.origin,
       a.opened_at, a.last_seen_at, a.details
from public.alerts a
join public.robots rb on rb.id = a.robot_id
where a.state in ('open', 'acknowledged');

grant select on public.v_readings_interpreted, public.v_latest_readings,
                public.v_robot_status, public.v_active_alerts to authenticated;
```

05_rls.sql — Políticas Row Level Security (reader vs technician)

```
-- ================================================================
-- CENTINELA - 05_rls.sql
-- Seguridad: lectura solo para miembros activos del domicilio.
-- Escritura de telemetria: SOLO via ingest_telemetry (service_role).
```

05_rls.sql — Políticas Row Level Security (reader vs technician) (continuación)

```
-- ====================================================================

-- Funciones auxiliares (security definer para evitar recursion de RLS)
create or replace function public.is_home_member(p_home_id uuid)
returns boolean language sql stable security definer set search_path = public as $$
    select exists (select 1 from public.memberships m
                   where m.home_id = p_home_id and m.user_id = auth.uid() and m.status = 'active');
$$;

create or replace function public.is_home_technician(p_home_id uuid)
returns boolean language sql stable security definer set search_path = public as $$
    select exists (select 1 from public.memberships m
                   where m.home_id = p_home_id and m.user_id = auth.uid()
                     and m.status = 'active' and m.role = 'technician');
$$;

create or replace function public.can_read_robot(p_robot_id uuid)
returns boolean language sql stable security definer set search_path = public as $$
    select exists (select 1 from public.robots r
                   where r.id = p_robot_id and public.is_home_member(r.home_id));
$$;

create or replace function public.can_read_frame(p_frame_id uuid)
returns boolean language sql stable security definer set search_path = public as $$
    select exists (select 1 from public.telemetry_frames f
                   where f.id = p_frame_id and public.can_read_robot(f.robot_id));
$$;

-- Los tecnicos reconocen alertas desde web/movil con: supabase.rpc('ack_alert', { p_alert_id })
create or replace function public.ack_alert(p_alert_id uuid)
returns boolean language plpgsql security definer set search_path = public as $$
declare v_home uuid;
begin
    select rb.home_id into v_home from public.alerts a
      join public.robots rb on rb.id = a.robot_id where a.id = p_alert_id;
    if v_home is null or not public.is_home_technician(v_home) then
        raise exception 'sin permiso para reconocer esta alerta';
    end if;
    update public.alerts set state = 'acknowledged'
     where id = p_alert_id and state = 'open';
    return found;
end;
$$;

grant execute on function public.is_home_member(uuid), public.is_home_technician(uuid),
      public.can_read_robot(uuid), public.can_read_frame(uuid), public.ack_alert(uuid)
      to authenticated;

-- RLS en todas las tablas
alter table public.homes             enable row level security;
alter table public.memberships       enable row level security;
alter table public.robots            enable row level security;
alter table public.devices           enable row level security;
alter table public.device_sessions   enable row level security;
alter table public.map_versions      enable row level security;
alter table public.missions          enable row level security;
alter table public.telemetry_frames  enable row level security;
alter table public.channel_catalog   enable row level security;
alter table public.channels          enable row level security;
alter table public.sensor_readings   enable row level security;
```

05_rls.sql — Políticas Row Level Security (reader vs technician) (continuación)

```
alter table public.frame_poses       enable row level security;
alter table public.robot_states      enable row level security;
alter table public.battery_readings  enable row level security;
alter table public.alerts            enable row level security;
alter table public.gas_calibrations  enable row level security;

-- Politicas de LECTURA (to authenticated)
drop policy if exists p_homes_read on public.homes;
create policy p_homes_read on public.homes for select to authenticated
    using (public.is_home_member(id));

drop policy if exists p_memberships_read on public.memberships;
create policy p_memberships_read on public.memberships for select to authenticated
    using (user_id = auth.uid());

drop policy if exists p_robots_read on public.robots;
create policy p_robots_read on public.robots for select to authenticated
    using (public.is_home_member(home_id));

drop policy if exists p_devices_read on public.devices;
create policy p_devices_read on public.devices for select to authenticated
    using (public.can_read_robot(robot_id));

drop policy if exists p_sessions_read on public.device_sessions;
create policy p_sessions_read on public.device_sessions for select to authenticated
    using (exists (select 1 from public.devices d
                   where d.id = device_id and public.can_read_robot(d.robot_id)));

drop policy if exists p_maps_read on public.map_versions;
create policy p_maps_read on public.map_versions for select to authenticated
    using (public.is_home_member(home_id));

drop policy if exists p_missions_read on public.missions;
create policy p_missions_read on public.missions for select to authenticated
    using (public.can_read_robot(robot_id));

drop policy if exists p_frames_read on public.telemetry_frames;
create policy p_frames_read on public.telemetry_frames for select to authenticated
    using (public.can_read_robot(robot_id));

drop policy if exists p_catalog_read on public.channel_catalog;
create policy p_catalog_read on public.channel_catalog for select to authenticated
    using (true);

drop policy if exists p_channels_read on public.channels;
create policy p_channels_read on public.channels for select to authenticated
    using (public.can_read_robot(robot_id));

drop policy if exists p_readings_read on public.sensor_readings;
create policy p_readings_read on public.sensor_readings for select to authenticated
    using (public.can_read_frame(frame_id));

drop policy if exists p_poses_read on public.frame_poses;
create policy p_poses_read on public.frame_poses for select to authenticated
    using (public.can_read_frame(frame_id));

drop policy if exists p_states_read on public.robot_states;
create policy p_states_read on public.robot_states for select to authenticated
    using (public.can_read_frame(frame_id));
```

05_rls.sql — Políticas Row Level Security (reader vs technician) (continuación)

```
drop policy if exists p_battery_read on public.battery_readings;
create policy p_battery_read on public.battery_readings for select to authenticated
    using (public.can_read_frame(frame_id));

drop policy if exists p_alerts_read on public.alerts;
create policy p_alerts_read on public.alerts for select to authenticated
    using (public.can_read_robot(robot_id));

drop policy if exists p_cal_read on public.gas_calibrations;
create policy p_cal_read on public.gas_calibrations for select to authenticated
    using (robot_id is null or public.can_read_robot(robot_id));
```